Les conditions en Python

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Les bases du test conditionnel **4**

A. Les bases du test conditionnel 4

B. Exercice : Quiz 6

III - Les tests multiples **9**

A. Les tests multiples 9

B. Exercice : Quiz 11

IV - L'essentiel **13**

V - Auto évaluation **14**

A. Exercice 14

B. Test 14

Solutions des exercices **16**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée : 1 heure

Environnement de travail : Thonny

Prérequis : aucun

Contexte

En informatique, nous traitons en permanence des données. Cela signifie qu'à partir d'informations, nous réalisons automatiquement des calculs, et en fonction des résultats nous prenons des décisions. Nous utilisons très souvent des structures de contrôle comme le `if` que nous allons voir pour effectuer des embranchements, c'est-à-dire prendre en compte les différents chemins possibles pour le traitement des données. L'exemple le plus simple est le test conditionnel binaire. Par exemple, pour accéder à ce cours, nous pouvons formaliser la situation suivante : si « *vous connaissez le principe des variables et les types de données* », alors « *Accédez à ce cours* », sinon « *Veillez à vous former aux prérequis* ». Nous avons donc une condition (formulée après le « *si* ») et 2 chemins possibles (formulés après le « *alors* » et le « *sinon* ») : le premier est emprunté si la condition est satisfaite, tandis que le second est emprunté si la condition n'est pas satisfaite.

Tous droits réservés à STUDI - Reproduction interdite

II Les bases du test conditionnel

A. Les bases du test conditionnel

Les valeurs booléennes

Az Définition

Le langage Python contient les valeurs booléennes *True* et *False*. True signifie « *Vrai* » en anglais, tandis que False correspond à « *Faux* ». Ces notions ont été travaillées par George Boole au XIXe siècle en Angleterre. Il est considéré comme le père de la logique moderne et son nom est désormais utilisé comme adjectif en informatique en son honneur. Vous serez amené à utiliser des variables « *booléennes* », c'est-à-dire des variables qui ont comme valeurs True ou False.

Les expressions booléennes

En informatique comme en mathématiques, nous appelons « *expression* » une ligne de calcul. La plupart du temps, ces expressions sont littérales, c'est-à-dire qu'elles contiennent des variables. Par exemple « *3 + a = 4* » est une expression littérale, dans laquelle « *a* » est une variable. Si cette variable vaut 1 alors l'égalité est correcte, alors que si cette variable est différente de la valeur 1, l'égalité est incorrecte. Nous ne pouvons pas savoir à l'avance si une expression sera correcte ou incorrecte. Pour le savoir, l'expression doit être **évaluée**, c'est-à-dire qu'elle doit être calculée en tenant compte des valeurs des variables.

Une expression littérale qui est évaluée avec True ou False est une expression booléenne. Pour évaluer des expressions booléennes, il nous faut découvrir les opérateurs de comparaison.

Les opérateurs de comparaison

Az Définition

Les opérateurs de comparaison basiques sont les suivants :

- a == b : compare l'égalité des expressions a et b, et est évalué à True uniquement si a est égal à b, sinon False.
- a != b : compare la non-égalité des expressions a et b, et est évalué à True uniquement si a et b ne sont pas égaux. Si a est égal à b, cette comparaison est évaluée à False.
- a < b : compare si a est plus petit que b, et est évalué à True si a est inférieur à b. Si a est plus grand que b, ou égal à b, cette comparaison est évaluée à False.
- a <= b : compare si a est plus petit ou égal à b, et est évalué à True si a est inférieur ou égal à b. Si a est strictement plus grand que b, cette comparaison est évaluée à False.
- a > b : compare si a est plus grand que b, et est évalué à True si a est supérieur à b. Si a est plus petit que b, ou égal à b, cette comparaison est évaluée à False.
- a >= b : compare si a est plus grand ou égal à b, et est évalué à True si a est supérieur ou égal à b. Si a est strictement plus petit que b, cette comparaison est évaluée à False.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Exécutez ces quelques lignes pour vous initier et écrivez-en d'autres pour vous habituer à ces opérateurs :

```
1 print(2==2) # affiche le résultat de l'évaluation de la comparaison de l'égalité entre 2 et 2
2 print(2>2) # affiche le résultat de l'évaluation de la comparaison de supériorité entre 2 et 2
3 print(2!=3) # affiche le résultat de l'évaluation de la comparaison de non-égalité entre 2 et 3
4 print(2<=1) # affiche le résultat de l'évaluation de la comparaison d'infériorité ou d'égalité entre 2 et 1
```

Les comparaisons par type de données

Attention

L'utilisation de ces opérateurs de comparaison peut sembler assez évidente de premier abord, car nous avons appris ces notions à l'école primaire avec les nombres entiers, puis au collège avec les nombres décimaux. Cependant, vous serez amené à comparer d'autres choses que des nombres entiers et des nombres décimaux. Nous vous invitons à prendre quelques secondes pour vous poser les questions suivantes et y réfléchir :

- Que signifie comparer 2 chaînes de caractères ?
- Que faire dans le cas où une chaîne de caractères est plus petite qu'une autre ?
- Que signifie comparer 2 listes, ou 2 tableaux ?
- Que faire dans le cas où une liste est plus petite qu'une autre ?

Vous vous rendez compte que la comparaison n'est plus aussi aisée qu'avec les nombres entiers.

Des comparaisons de chaînes de caractères et de listes

Exemple

La comparaison des chaînes de caractères s'opère selon l'ordre **lexicographique**. Il s'apparente à l'ordre alphabétique d'un dictionnaire. Dans un dictionnaire, les mots commençant par la lettre « *a* » sont ordonnés avant les mots commençant par la lettre « *b* ». Les mots commençant par « *a* » sont donc plus petits que les mots commençant par « *b* ». Ainsi, nous avons : "programme" < "python" (lire « *plus petit que* », car la lettre « *y* » est ordonnée après la lettre « *r* » dans le dictionnaire).

Cependant, l'ordre lexicographique diffère de l'ordre alphabétique. Par exemple, il y a une distinction des lettres majuscules et des lettres minuscules : les majuscules sont situées avant les minuscules. Ainsi, le mot « *Python* » (avec une majuscule) est ordonné avant le mot « *python* » (sans majuscule). Une chaîne de caractères peut également contenir des caractères qui ne sont pas des lettres, mais des chiffres ou encore des symboles : la plupart de ces caractères non lettrés sont ordonnés avant les lettres.

Concernant les listes (également appelées « *séquences* »), c'est là encore l'ordre lexicographique qui est utilisé pour les comparaisons. Ainsi, nous avons : [1,2,3] > [1,0,2,4] (car 2 est plus grand que 0).

Tous droits réservés à STUDI - Reproduction interdite

Introduction au « if »

Dans un programme informatique, les traitements effectués dépendent des valeurs des variables. Le programmeur code les différents scénarios possibles pour que son programme fasse ce qui est attendu. Le programmeur teste ses variables (à l'aide des opérateurs de comparaison) dans des expressions booléennes. En fonction du résultat (True ou False), il code un traitement ou pas. Ce test s'effectue avec le mot-clé `if` suivi de l'expression booléenne à évaluer entre parenthèses, elle-même suivie du caractère `:`. Les lignes situées en dessous et indentées sont exécutées uniquement si l'expression booléenne est évaluée à True. En anglais, *if* signifie « *si* » en français. Voyons le modèle de la syntaxe avec l'exemple présenté plus haut.

```
1 if ("programme"<"python"): # cette ligne évalue l'expression de comparaison à True
2 # car « programme » et ordonné avant « python »
3   print("'programme' est plus petit que 'python'") # cette ligne est alors exécutée
```

Maintenant que vous connaissez la syntaxe de base du `if` et le principe des expressions booléennes, nous vous invitons à tester le programme interactif suivant. Arriverez-vous à obtenir (en plusieurs exécutions) les 3 affichages prévus ? Si vous n'y arrivez pas, saisissez « *pierre* », « *xavier* », puis « *python* ».

```
1 entree = input("Saisissez votre prénom, puis validez avec la touche 'Entrée' : ")
2 # la ligne ci-dessus va demander une saisie à l'utilisateur et l'enregistrer dans la
  variable 'entree'
3 # notez que le résultat de la fonction « input » est de type chaîne de caractères
  (string)
4 if (entree<"python"): # cette expression vaut True si le mot saisi est ordonné avant
  le mot 'python'
5   print("Votre prénom est plus petit que 'python'") # cette ligne s'exécute dans le
  cas ci-dessus
6 if (entree>"python"): # cette expression vaut True si le mot saisi est ordonné après
  le mot 'python'
7   print("Votre prénom est plus grand que 'python'") # cette ligne s'exécute dans le
  cas ci-dessus
8 if (entree=="python"): # cette expression vaut True si le mot saisi est le mot
  'python'
9   print("Votre prénom est 'python' !") # cette ligne s'exécute dans le cas ci-dessus
```

Attirons maintenant votre attention sur le fait que l'expression après le `if` est évaluée comme une expression booléenne, même si elle n'en est pas vraiment une. Un objet vide est évalué à False, tandis qu'un objet non vide est évalué à True. Voici quelques exemples pour vous en rendre compte :

```
1 if ("une chaine"): # une chaîne de caractères non vide, sans comparaison, est évaluée
  à True
2   print("une chaîne de caractères n'est pas False, donc considérée comme True après
  un 'if' !")
3 if (5): # un simple nombre entier, sans comparaison, est évalué à True
4   print("un nombre entier n'est pas False, donc considéré comme True après un 'if'
  !")
5 if ([2]): # une liste non vide, sans comparaison, est évaluée à True
6   print("une liste n'est pas False, donc considérée comme True après un 'if' !")
7 if (""): # une chaîne de caractères vide, sans comparaison, est évaluée à False
8   print("une chaîne de caractères vide est évaluée à False, donc cette ligne n'est
  jamais affichée !")
9 if ([]): # une liste vide, sans comparaison, est évaluée à False
10   print("une liste vide est évaluée à False, donc cette ligne n'est jamais affichée
  !")
```

B. Exercice : Quiz

[solution n°1 p. 16]

Question 1

Une variable booléenne ne peut avoir que 2 seules valeurs possibles.

Tous droits réservés à STUDI - Reproduction interdite

Vrai

Faux

Question 2

L'exécution du code `print(52 != 53)` affichera :

True

False

Une erreur

Question 3

L'exécution du code `print(["Python",3.10,"apprenant","à distance"] < ["Python",3.10,"apprentissage"])` affichera :

True

False

Une erreur

Question 4

Parmi les expressions suivantes, laquelle n'est pas une expression booléenne, en sachant que A et B sont des expressions booléennes ?

A <= B

A != B

A = B

Question 5

Un programme commence avec la ligne de code `age = input("Indiquez votre âge : ")`. Parmi les 3 propositions suivantes, indiquez la suite du programme qui ne générera pas d'erreur.

```
if (age>18)       print("Vous êtes majeur")
```

```
if (age>18):       print("Vous êtes majeur")
```

Tous droits réservés à STUDI - Reproduction interdite

```
if (age>"18"):       print("Vous êtes majeur")
```


Tous droits réservés à STUDI - Reproduction interdite

III Les tests multiples

A. Les tests multiples

Les « if » imbriqués

Nous avons vu que le `if` permet de tester une expression booléenne et de déclencher un traitement particulier. Mais en programmation, nous avons souvent plusieurs expressions à évaluer à la suite (ou en même temps). Nous pouvons alors imbriquer les `if`. Dans ce cas, il faut faire encore plus attention à l'indentation de votre code. Exécutez le code suivant plusieurs fois pour faire apparaître les 2 affichages prévus (la ligne « *Les 3 nombres saisis sont plus petits ou égaux à 5.* » s'affiche ou ne s'affiche pas).

```
1 # on demande 3 saisies à l'utilisateur, que l'on convertit en type entier
2 # ces saisies, une fois converties, sont mémorisées dans les variables nb1, nb2, nb3
3 nb1 = int(input("Saisissez un premier nombre entre 0 et 9 :"))
4 nb2 = int(input("Saisissez un deuxième nombre entre 0 et 9 :"))
5 nb3 = int(input("Saisissez un troisième nombre entre 0 et 9 :"))
6 if (nb1<=5):
7   # ce code est exécuté si le premier nombre saisi est strictement inférieur à 6
8   if(nb2<=5):
9     # ce code est exécuté si le deuxième nombre saisi est strictement inférieur à 6
10    if (nb3<=5):
11      # ce code est exécuté si le troisième nombre saisi est strictement inférieur à 6
12      print("Les 3 nombres saisis sont plus petits ou égaux à 5.")
13 print("Le programme est terminé.") # cette ligne s'affiche toujours, regardez bien son indentation
```

Les opérateurs logiques

Az Définition

Dans l'algèbre de Boole, que l'on peut aussi appeler « *calcul binaire* », sont définies 4 fonctions logiques :

- « *ou* » en français, `or` en anglais et en Python,
- « *et* » en français, `and` en anglais et en Python,
- « *non* » en français, `not` en anglais et en Python,
- « *ou exclusif* » en français, appelé traditionnellement « *xor* » et noté ^ en Python.

Ces fonctions sont définies ainsi :

- `(A or B)` est Vrai, si A est Vrai ou B est Vrai (il suffit d'un seul sur les deux).
- `(A and B)` est Vrai si A est Vrai et B est Vrai (les deux doivent être Vrai).
- `(not A)` est Vrai, le contraire de A.
- `(A ^ B)` est Vrai si A est Vrai ou B est Vrai, mais pas les deux ensembles.

Ces opérateurs logiques peuvent être utilisés dans les expressions booléennes, en particulier dans la condition du `if`. Comparez le code suivant avec le précédent, dont le fonctionnement est exactement identique.

```
1 nb1 = int(input("Saisissez un premier nombre entre 0 et 9 :"))
2 nb2 = int(input("Saisissez un deuxième nombre entre 0 et 9 :"))
3 nb3 = int(input("Saisissez un troisième nombre entre 0 et 9 :"))
4 if (nb1<=5 and nb2<=5 and nb3<=5): # les 3 tests sont regroupés dans un unique « if »
5   print("Les 3 nombres saisis sont plus petits ou égaux à 5.")
6 print("Le programme est terminé.")
```

Tous droits réservés à STUDI - Reproduction interdite

Vous pouvez tester les différents opérateurs logiques en modifiant le code ci-dessus dans votre interpréteur Python afin de bien comprendre, par la manipulation, comment ils fonctionnent.

Le « else »

Grâce au `if`, vous savez maintenant effectuer des traitements spécifiques lorsque le test conditionnel est évalué à True. Mais très souvent, nous avons besoin également d'effectuer des traitements spécifiques lorsque le test conditionnel est évalué à False. Pour cela nous utilisons le mot-clé `else`, qu'on peut traduire en français par « *sinon* ». Voyons la syntaxe maintenant :

```
1 age = int(input("Indiquez votre âge : ")) # on demande de saisir un nombre, converti
  en entier
2 if (age>=18): # le test conditionnel compare la variable 'age' au nombre 18
3   print("Vous êtes majeur.") # cette ligne s'affiche si la condition est évaluée à
  True
4 else:
5   print("Vous êtes mineur.") # cette ligne s'affiche si la condition est évaluée à
  False
```

Comme vous pouvez le constater, le `else` est aligné avec la même indentation que le `if`. Le caractère `:` termine la ligne du `else`. Les lignes en dessous et indentées sont exécutées uniquement lorsque le test conditionnel du `if` n'est pas Vrai.

Inverser la condition

⚠ Attention

Il peut arriver que l'on souhaite effectuer un test conditionnel, et ne pas effectuer de traitement dans le cas où il est évalué à Vrai, mais en effectuer dans le `else`. On serait tenté d'utiliser cette syntaxe :

```
1 if (age>=18):
2   # on ne fait rien ici
3 else:
4   print("Vous êtes trop jeune.")
```

Mais cela est incorrect : il doit toujours y avoir des traitements spécifiques pour le cas Vrai. Il faut donc s'arranger pour inverser la condition. Dans notre exemple très simple, nous pourrions écrire :

```
1 if (age<18):
2   print("Vous êtes trop jeune.")
```

Le « elif »

Le couple de mots-clés `if` / `else` est très utile. Mais dans certains cas de figure, les tests à effectuer ne sont pas aussi binaires. Il arrive parfois que l'on souhaite effectuer plusieurs tests conditionnels et dans chaque cas, effectuer des traitements particuliers. Pour cela, on utilise le mot-clé `elif`, qui est une contraction de `else` et `if`, que l'on peut traduire en français par « *sinon, si* ». Pour utiliser le `elif`, il est nécessaire de fournir un autre test conditionnel. Il est possible d'avoir plusieurs `elif` après un `if`. Enfin, les traitements codés dans un éventuel `else` seront exécutés si aucune condition n'a été satisfaite. Voyons un exemple maintenant.

```
1 entree = input("Saisissez votre prénom : ")
2 if (entree<"python"):
3   print("Votre prénom est plus petit que 'python'")
4 elif (entree>"python"): # ce test est évalué seulement si le précédent a été évalué à
  False
5   print("Votre prénom est plus grand que 'python'")
6 elif (entree=="python"): # ce test est évalué seulement si les précédents ont été
  évalués à False
7   print("Votre prénom est 'python' !")
```


Tous droits réservés à STUDI - Reproduction interdite

```
8 else: # si tous les tests ont été évalués à False, alors on exécute les lignes ci-dessous
9   print("Erreur de saisie de votre prénom !")
```

Normalement, vous ne devriez pas voir la dernière ligne s'afficher (« *Erreur de saisie* ») lors de votre exécution de ce programme. Cette ligne a été écrite pour illustrer la syntaxe, mais ne sera jamais exécutée, car les 3 tests couvrent tous les cas possibles.

B. Exercice : Quiz

[solution n°2 p. 18]

Question 1

Laquelle des expressions booléennes suivantes affichera True ?

```
print( (True or False) and False or (True ^ True) )
```

```
print( (True ^ False) or False and (True or True) )
```

```
print( (True and False) or False and (True and True) )
```

Question 2

Le code suivant est-il correct ?

```
1 if (14<age<18) :
2   print("Vous bénéficiez du tarif adolescent.")
```

Oui

Non

Question 3

Il vaut mieux prévoir le « *else* » quand on code un « *if* », même si on y met aucune ligne de traitement dans l'immédiat.

Vrai

Faux

Question 4

On peut imbriquer autant de « *if / else* » que l'on veut.

Vrai

Faux

Question 5

Les expressions booléennes des « *elif* » doivent tester la même variable que pour le « *if* ».

Tous droits réservés à STUDI - Reproduction interdite

Vrai

Faux

Tous droits réservés à STUDI - Reproduction interdite

IV L'essentiel

Vous avez appris la structure du « *if / elif / else* » qui permet d'effectuer des tests conditionnels et d'y associer des traitements spécifiques. La base du test conditionnel consiste en l'évaluation d'expressions booléennes, qui peuvent être parfois complexes et qui requièrent l'utilisation d'opérateurs logiques. Le résultat des expressions booléennes dépend des types de données utilisées : entendre des chaînes de caractères comme des nombres n'est pas logique. Chaque traitement spécifique à un test conditionnel est implémenté avec des blocs d'instructions. En Python, les blocs d'instructions doivent bien respecter l'indentation ; cela demande d'autant plus de vigilance lorsque le code est très imbriqué. Pour rendre votre code lisible, il est recommandé d'y écrire des commentaires (en utilisant le caractère #).

Tous droits réservés à STUDI - Reproduction interdite

V Auto évaluation

A. Exercice

Les services de météorologie diffusent des informations importantes pour la santé des populations, par exemple les températures lors de canicules estivales. L'objectif est d'avoir un programme informatique qui diffuse des conseils à la population en fonction de la température donnée. Dans ce cas pratique, vous devez considérer 5 zones de confort (des intervalles de températures) basées en France métropolitaine.

Question 1

[solution n°3 p. 19]

Rédigez un texte qui explique votre démarche pour concevoir ce programme.

Question 2

[solution n°4 p. 19]

En vous inspirant de tout ce que vous avez vu dans ce cours, écrivez un programme qui répond à la problématique posée.

B. Test

Exercice 1 : Quiz

[solution n°5 p. 20]

Question 1

Lorsqu'on effectue une comparaison dans un test conditionnel, il faut veiller à ce que les 2 expressions de chaque côté de l'opérateur de comparaison soient de même type.

- Vrai
- Faux

Question 2

Dans l'expression booléenne (A or B and C), nous savons que A vaut True. Que vaut l'expression entière ?

- A
- B
- C

Question 3

Le programme suivant est-il correct ?

```
1 jour = int(input("Entrez la date du jour (nombre entre 1 et 31 compris) : "))
2 if (jour<15):
3   print("Nous sommes dans la première quinzaine.")
4 else:
5   print("Nous sommes dans la deuxième quinzaine.")
```


Tous droits réservés à STUDI - Reproduction interdite

Oui

Non

Question 4

Nous avons amélioré le programme de la question précédente pour signaler à l'utilisateur une erreur de saisie. Ce nouveau programme est-il correct ?

```
1 jour = int(input("Entrez la date du jour (nombre entre 1 et 31 compris) : "))
2 if (jour<1 and jour>31):
3   print("Vous avez fait une erreur de saisie !")
4 elif (jour<15):
5   print("Nous sommes dans la première quinzaine.")
6 else:
7   print("Nous sommes dans la deuxième quinzaine.")
```

Oui

Non

Question 5

Le programme suivant est-il correct ?

```
1 jour = int(input("Entrez la date du jour (nombre entre 1 et 31 compris) : "))
2 mois = int(input("Entrez la date du mois (nombre entre 1 et 12 compris) : "))
3 if (jour>30):
4   if (mois==2 or mois==4 or mois==6 or mois==9 or mois==11):
5     # les mois de février, avril, juin, septembre, novembre n'ont pas plus de 30
  jours
6     print("Vous avez fait une erreur de saisie !")
7   else :
8     print("C'est le dernier jour du mois.")
9 elif (jour>29 and mois=2):
10   # en février il n'y a pas plus de 29 jours
11   print("Vous avez fait une erreur de saisie !")
12 else:
13   print("Vous avez saisi un jour valide.")
```

Oui

Non

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 6]

Question 1

Une variable booléenne ne peut avoir que 2 seules valeurs possibles.

- ✓ Vrai
- Faux

Une variable booléenne ne peut avoir que la valeur True ou False, soit 2 valeurs possibles uniquement.

Question 2

L'exécution du code `print(52 != 53)` affichera :

- ✓ True
- False
- Une erreur

L'opérateur `!=` se lit « *différent* » ou encore « *non égal* ». Comme le nombre 52 est différent du nombre 53, l'expression est vraie et Python affiche True.

Question 3

L'exécution du code `print(["Python",3.10,"apprenant","à distance"] < ["Python",3.10,"apprentissage"])` affichera :

- ✓ True
- False
- Une erreur

Tous droits réservés à STUDI - Reproduction interdite

La comparaison de listes utilise l'ordre lexicographique, c'est-à-dire que la comparaison s'effectue terme à terme. Les 2 premiers termes des 2 listes sont identiques, mais le troisième terme est différent. Dans un dictionnaire, le mot « *apprenant* » est situé avant le mot « *apprentissage* » donc le mot « *apprenant* » est plus petit que le mot « *apprentissage* ». La première liste est donc plus petite que la seconde.

Question 4

Parmi les expressions suivantes, laquelle n'est pas une expression booléenne, en sachant que A et B sont des expressions booléennes ?

A <= B

A != B

✓ A = B

Les opérateurs « *<=* » et « *!=* » sont des opérateurs booléens donc il s'agit d'expressions booléennes. Mais l'opérateur « *=* » est l'opérateur d'assignation (également appelé affectation), qui est parfois confondu avec l'opérateur de comparaison « *==* ».

Question 5

Un programme commence avec la ligne de code `age = input("Indiquez votre âge : ")`. Parmi les 3 propositions suivantes, indiquez la suite du programme qui ne générera pas d'erreur.

```
if (age>18)       print("Vous êtes majeur")
```

```
if (age>18):       print("Vous êtes majeur")
```

```
✓ if (age>"18"):       print("Vous êtes majeur")
```

La première proposition indiquera une erreur de syntaxe, car il manque le caractère « *:* » au bout de la ligne du « *if* ». La deuxième proposition indiquera une erreur lors de l'exécution, car nous voulons comparer 2 données de type différent. En effet, la variable « *age* » contient une chaîne de caractères (string) alors que « *18* » est un nombre entier (integer). La dernière proposition ne déclenche pas d'erreur. Néanmoins, ce code ne va pas faire ce que l'on peut supposer, car la comparaison des chaînes de caractères fonctionne selon l'ordre lexicographique, donc la phrase « *Vous êtes majeur* » sera affichée aussi pour les âges saisis de 2 à 9 !

Tous droits réservés à STUDI - Reproduction interdite

studi

Solution n°2

[exercice p. 11]

Question 1

Laquelle des expressions booléennes suivantes affichera True ?

```
print( (True or False) and False or (True ^ True) )
```

✓ `print( (True ^ False) or False and (True or True) )`

```
print( (True and False) or False and (True and True) )
```

Il convient d'évaluer d'abord les sous-expressions entre parenthèses, puis de gauche à droite. La première proposition se simplifie en (True and False or False), puis en (False or False). La deuxième proposition se simplifie en (True or False and True), puis en (True and True). La dernière proposition se simplifie en (False or False and True), puis en (False and True).

Question 2

Le code suivant est-il correct ?

```
1 if (14<age<18) :
2   print("Vous bénéficiez du tarif adolescent.")
```

Oui

✓ Non

L'expression booléenne est mal formée. Elle pourrait plutôt s'écrire `(age > 14 and age < 18)`.

Question 3

Il vaut mieux prévoir le « *else* » quand on code un « *if* », même si on y met aucune ligne de traitement dans l'immédiat.

Vrai

✓ Faux

De la même façon que nous ne pouvons pas avoir un « *if* » sans code si le test est Vrai, nous ne pouvons pas avoir de « *else* » sans code. Python indiquerait une erreur de syntaxe. Il ne faut donc pas écrire de « *else* » si vous n'avez rien à coder là.

Tous droits réservés à STUDI - Reproduction interdite

Question 4

On peut imbriquer autant de « *if / else* » que l'on veut.

✓ Vrai

Faux

En effet, il n'y a pas de limitation particulière, mais soyez vigilant à l'indentation et veillez à garder votre code lisible.

Question 5

Les expressions booléennes des « *elif* » doivent tester la même variable que pour le « *if* ».

Vrai

✓ Faux

Il n'y a pas de contrainte particulière entre ces expressions booléennes. Mais dans les faits, le plus souvent les différentes valeurs possibles d'une même variable sont testées.

[exercice p. 14]

Solution n°3

Pour commencer, il faudra demander de saisir la température et de mémoriser la valeur saisie dans une variable en tant que nombre entier (pas une chaîne de caractères). Ensuite, nous devons tester plusieurs niveaux de température, et pour chacun afficher une recommandation appropriée. Il est demandé de considérer 5 zones de confort, donc nous décidons des 5 paliers suivants (les valeurs précises ne sont pas importantes, l'important est la démarche) :

- Températures supérieures à 35 °C,
- Températures comprises entre 25 °C et 35 °C,
- Températures comprises entre 15 °C et 25 °C,
- Températures comprises entre 5 °C et 15 °C,
- Températures inférieures à 5 °C.

[exercice p. 14]

Solution n°4

Vous avez probablement utilisé l'opérateur logique and pour effectuer vos tests conditionnels afin d'encadrer les différents niveaux de température. Si c'est le cas, sachez que cela est inutile. Découvrez une solution utilisant `elif` :

```
1 temperature = int(input("Indiquez la température : "))
2 if (temperature>35):
3   print("Veillez à vous protéger de la chaleur en recherchant les ilots de
  fraîcheur.")
4 elif (temperature>25):
5   print("Veillez à vous hydrater suffisamment.")
```

Tous droits réservés à STUDI - Reproduction interdite

```
elif (temperature>15):
  print("Passez une bonne journée.")
elif (temperature>5):
  print("Veillez à vous couvrir suffisamment pour ne pas avoir froid.")
else:
  # dans ce cas la température est inférieur à 5° (le caractère dièse indique un commentaire)
  print("Attention aux plaques de verglas et aux chutes de neige.")
```

Solution n°5

[exercice p. 14]

Question 1

Lorsqu'on effectue une comparaison dans un test conditionnel, il faut veiller à ce que les 2 expressions de chaque côté de l'opérateur de comparaison soient de même type.

- ✓ Vrai
- Faux

En effet, si on compare des objets de type différent, une erreur sera déclenchée lors de l'exécution du programme.

Question 2

Dans l'expression booléenne (A or B and C), nous savons que A vaut True. Que vaut l'expression entière ?

- ✓ A
- B
- C

Comme A vaut True, l'expression (A or B and C) revient à évaluer True or (B and C). L'opérateur or retourne immédiatement True, sans avoir besoin d'évaluer (B and C). De plus en python, and est évaluer avant le or. L'expression entière vaut donc A.

Question 3

Le programme suivant est-il correct ?

```
jour = int(input("Entrez la date du jour (nombre entre 1 et 31 compris) : "))
if (jour<15):
  print("Nous sommes dans la première quinzaine.")
else:
  print("Nous sommes dans la deuxième quinzaine.")
```


Tous droits réservés à STUDI - Reproduction interdite

✓ Oui

Non

En effet, ce programme ne présente aucune erreur de syntaxe ou de fonctionnement. Néanmoins, même s'il est précisé à l'utilisateur de saisir un nombre compris entre 1 et 31, il serait plus prudent d'envisager les cas où le nombre saisi est en dehors de cet intervalle. Essayez donc ce programme en indiquant 40 ou encore - 2, vous verrez que ce code peut aboutir à des situations erronées.

Question 4

Nous avons amélioré le programme de la question précédente pour signaler à l'utilisateur une erreur de saisie. Ce nouveau programme est-il correct ?

```
1 jour = int(input("Entrez la date du jour (nombre entre 1 et 31 compris) : "))
2 if (jour<1 and jour>31):
3   print("Vous avez fait une erreur de saisie !")
4 elif (jour<15):
5   print("Nous sommes dans la première quinzaine.")
6 else:
7   print("Nous sommes dans la deuxième quinzaine.")
```

Oui

✓ Non

Une erreur s'est glissée : l'opérateur `and` devrait être remplacé par l'opérateur `or`. En effet, le jour saisi ne peut pas être en même temps inférieur à 1 et supérieur à 31, mais nous voulons que le message d'alerte s'affiche quand l'un ou l'autre est vrai.

Question 5

Le programme suivant est-il correct ?

```
1 jour = int(input("Entrez la date du jour (nombre entre 1 et 31 compris) : "))
2 mois = int(input("Entrez la date du mois (nombre entre 1 et 12 compris) : "))
3 if (jour>30):
4   if (mois==2 or mois==4 or mois==6 or mois==9 or mois==11):
5     # les mois de février, avril, juin, septembre, novembre n'ont pas plus de 30
  jours
6     print("Vous avez fait une erreur de saisie !")
7   else :
8     print("C'est le dernier jour du mois.")
9 elif (jour>29 and mois=2):
10   # en février il n'y a pas plus de 29 jours
11   print("Vous avez fait une erreur de saisie !")
12 else:
13   print("Vous avez saisi un jour valide.")
```

Tous droits réservés à STUDI - Reproduction interdite

Oui

✓ Non

En effet l'opérateur de comparaison d'égalité est == alors que sur la ligne du `elif` se trouve l'opérateur =.

Tous droits réservés à STUDI - Reproduction interdite